

Vendor Evidence Package

Module: pqc_crypto_module v0.1.0 **Date:** 2026-04-28

Disclaimer: This package does not imply FIPS 140-3 validation or CMVP certification.

1. Purpose

This document defines the vendor evidence package for lab submission, covering the software bill of materials (SBOM), dependency analysis, license compliance, vulnerability scanning, and third-party crypto assessment.

2. SBOM Plan

2.1 Tooling

Tool	Purpose	Command
cargo-cyclonedx	Generate CycloneDX SBOM in JSON/XML	cargo cyclonedx --format json --output-file sbom.cdx.json
cargo tree	Dependency tree visualization	cargo tree -p pqc_crypto_module
cargo metadata	Machine-readable dependency graph	cargo metadata --format-version=1

2.2 SBOM Contents

The SBOM will include for each dependency: - Package name and version - License (SPDX identifier) - Source repository URL - Cryptographic relevance flag (does this crate perform crypto operations?) - Integrity hash (from Cargo.lock)

2.3 Generation Schedule

- Generate SBOM at each tagged release

- Archive with build artifacts
- Include in lab submission package

3. Direct Dependencies

#	Crate	Version	Purpose	Crypto-relevant	License
1	pqcrypto-mlds	latest	ML-DSA-65 implementation (C backend)	Yes — primary	MIT/ Apache-2.0
2	pqcrypto-traits	latest	Trait definitions for PQC algorithms	Yes (traits only)	MIT/ Apache-2.0
3	sha3	latest	SHA3-256 (pure Rust, RustCrypto)	Yes	MIT/ Apache-2.0
4	sha2	latest	SHA-256 (non-approved, legacy)	Yes (non-approved)	MIT/ Apache-2.0
5	ed25519-dalek	latest	Ed25519 (non-approved, legacy)	Yes (non-approved)	BSD-3-Clause
6	hmac	latest	HMAC-SHA256 (non-approved, legacy)	Yes (non-approved)	MIT/ Apache-2.0
7	rand	0.8.x	RNG utilities	Yes (RNG)	MIT/ Apache-2.0
8	rand_core	0.6.x	OsRng, RNG traits	Yes (RNG)	MIT/ Apache-2.0
9	zeroize	1.x	Secure memory zeroization	Yes (key management)	MIT/ Apache-2.0
10	thiserror	latest	Error type derivation	No	MIT/ Apache-2.0
11	hex	latest	Hex encoding/decoding	No	MIT/ Apache-2.0

3.1 Transitive Crypto Dependencies

Crate	Pulled by	Purpose	Notes
pqcrypto-internals	pqcrypto-mlds	C compilation harness for PQC reference implementations	Compiles C code via cc crate; this is the actual

Crate	Pulled by	Purpose	Notes
			crypto implementation
cc	pqcrypto-internals	C compiler invocation	Build-time only
getrandom	rand_core	OS entropy access	Platform-specific syscalls
digest	sha3, sha2	Hash trait definitions	No crypto itself
crypto-common	digest	Common crypto trait types	No crypto itself
curve25519-dalek	ed25519-dalek	Elliptic curve arithmetic	Non-approved path only

4. License Review

4.1 Summary

License	Count	Compatible with proprietary use?
MIT	11	Yes
Apache-2.0	10	Yes
MIT/Apache-2.0 (dual)	9	Yes
BSD-3-Clause	1	Yes

Conclusion: All dependencies use permissive open-source licenses. No copyleft (GPL, LGPL, AGPL) licenses detected. No license conflicts for commercial or proprietary use.

4.2 License Verification

Verify with cargo-deny
cargo deny check licenses

Expected output: all licenses in allow list

Recommended deny.toml allow list:

```
[licenses]
allow = [
    "MIT",
    "Apache-2.0",
    "BSD-3-Clause",
    "BSD-2-Clause",
    "ISC",
```

```
"Unicode-DFS-2016",  
]
```

5. Vulnerability Scanning

5.1 cargo audit

```
# Install  
cargo install cargo-audit  
  
# Run against RustSec advisory database  
cargo audit  
  
# Expected: 0 vulnerabilities in direct or transitive dependencies
```

5.2 cargo deny

```
# Install  
cargo install cargo-deny  
  
# Check advisories, licenses, bans, and sources  
cargo deny check  
  
# Sections:  
#   advisories – known CVEs  
#   licenses – license compliance  
#   bans – disallowed crates  
#   sources – only crates.io allowed (no git dependencies)
```

5.3 Recommended deny.toml for Security

```
[advisories]  
vulnerability = "deny"  
unmaintained = "warn"  
yanked = "deny"  
  
[sources]  
unknown-registry = "deny"  
unknown-git = "deny"  
allow-registry = ["https://github.com/rust-lang/crates.io-index"]  
allow-git = []
```

5.4 Scanning Schedule

- Run `cargo audit` on every CI build
- Run `cargo deny check` before each release
- Subscribe to RustSec advisory notifications for all direct dependencies

- Review new advisories within 48 hours of publication
-

6. Third-Party Crypto Statement

6.1 pqcrypto-mldsa (Primary Crypto Dependency)

What it is: Rust bindings to the PQClean C reference implementation of ML-DSA (formerly CRYSTALS-Dilithium).

How it works: 1. pqcrypto-internals compiles the C reference implementation from PQClean sources at build time using the cc crate 2. pqcrypto-mldsa provides safe Rust wrappers around the C FFI 3. The C code performs the actual key generation, signing, and verification

Provenance: - PQClean project: <https://github.com/PQClean/PQClean> - Rust bindings: <https://github.com/rustpq/pqcrypto> - The C implementation tracks the NIST reference implementation

Implications for FIPS validation: - The actual cryptographic algorithm executes in compiled C code, not Rust - The C code is part of the module boundary (compiled into the same binary) - A lab may require review of the C source, not just the Rust wrappers - Algorithm correctness depends on PQClean tracking the final FIPS 204 specification

Risk assessment: - PQClean is a well-maintained project with multiple contributors and CI - The C code is reference-quality, not optimized (no AVX2/NEON paths by default) - Version pinning via Cargo.lock ensures reproducibility

6.2 RustCrypto Crates (sha3, sha2, hmac)

What they are: Pure Rust implementations of hash functions and MACs from the RustCrypto project.

Provenance: <https://github.com/RustCrypto>

Risk: Low. RustCrypto is widely used, audited, and maintained. SHA3-256 implementation has been reviewed against the NIST specification.

6.3 ed25519-dalek

What it is: Rust implementation of Ed25519 from the Dalek Cryptography project.

Relevance: Non-approved algorithm, used only in legacy mode. Not part of the FIPS-approved service set.

7. Maintenance Risk Assessment

Dependency	Last updated	Maintainer(s)	Bus factor	Risk
pqcrypto-mldsa	Active	rustpq team	2-3	MEDIUM — small team, niche domain
sha3	Active	RustCrypto org	5+	LOW
sha2	Active	RustCrypto org	5+	LOW
ed25519-dalek	Active	Dalek/Zcash team	3+	LOW
zeroize	Active	RustCrypto org	5+	LOW
rand / rand_core	Active	rust-random team	3+	LOW
thiserror	Active	dtolnay	1 (high-profile)	LOW
hex	Active	Community	2+	LOW
hmac	Active	RustCrypto org	5+	LOW

Primary risk: pqcrypto-mldsa is maintained by a small team. If the crate becomes unmaintained, the module would need to either (a) fork and maintain the C bindings, or (b) switch to an alternative ML-DSA implementation (e.g., ml-dsa from RustCrypto when stable).

Mitigation: Pin versions via Cargo.lock. Monitor crate activity quarterly. Maintain a fork-ready posture.

8. Evidence Checklist for Lab Submission

Artifact	Status	Tool
CycloneDX SBOM (JSON)	Not yet generated	cargo-cyclonedx
Dependency tree	Available via cargo tree	cargo tree
License report	Not yet generated	cargo deny check licenses
Vulnerability scan	Not yet run as formal evidence	cargo audit
Source ban verification	Not yet configured	cargo deny check bans
Third-party crypto provenance statement	This document (Section 6)	Manual

Artifact	Status	Tool
Cargo.lock snapshot	Committed in repository	Git

End of vendor evidence package.